

Multimodal: Images as Tokens

Teaching a language model to read pictures · video 2:18

This lesson answers one question: how does a language model read an image? By turning it into tokens. A tiny model built from scratch cuts a 32×32 image into 16 patches, projects each with one linear layer into the same space as the text tokens, and reads image and caption as one sequence. After 1,500 steps it describes new images with the right color and position every time, and the right shape 91% of the time; real vision-language models follow the same plan at scale.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 5).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. From pixels to tokens	▶ 0:08-0:55	2	☐
2. Training and testing	▶ 0:55-1:29	2	☐
3. Real models, and the cost	▶ 1:29-2:08	2	☐

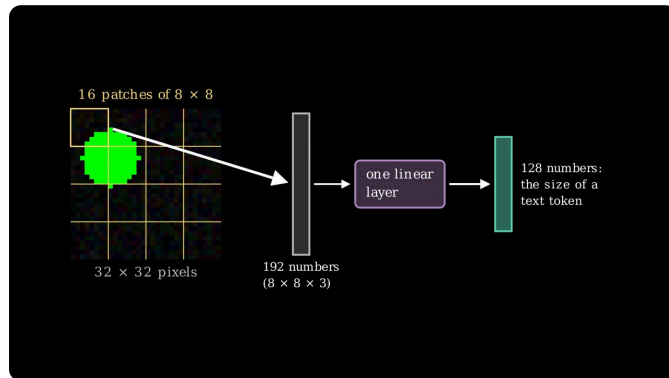
Before you start: How LLMs Work episodes 3-4 (tokens and embeddings) and Deep Dive episode 7 (the causal mask). Code: `code/d30_multimodal` (PyTorch only; no downloads; about 10 minutes on a CPU).

CONCEPT 1 OF 3

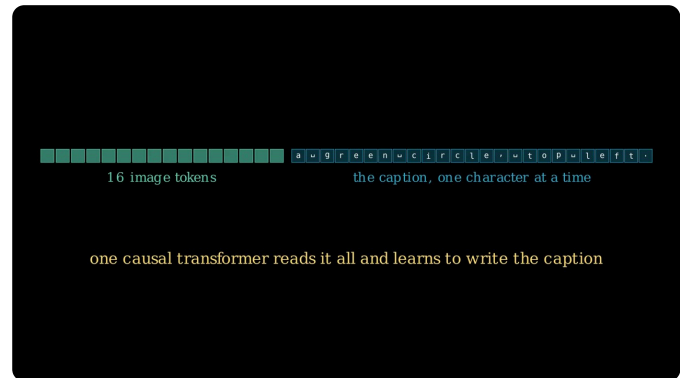
From pixels to tokens

▶ Video 0:08–0:55

A transformer only reads sequences of vectors, so the image is turned into tokens. A 32×32 color image is cut into **16 patches** of 8×8 pixels; each patch is $8 \times 8 \times 3 = 192$ numbers. One linear layer turns each patch into a vector of **128** numbers, exactly the size of a text token. Here that layer, 24,704 parameters, is the whole vision encoder.



▶ 0:40 16 patches of $8 \times 8 \times 3 = 192$ numbers; one linear layer maps each to a 128-number token.



▶ 0:54 One sequence: 16 image tokens, then the caption one character at a time.

Image and text then become **one sequence**: 16 image tokens followed by the characters of a caption. One causal transformer (830,103 parameters) reads it all and learns to write the caption, e.g. “a green circle, top left.”

KEY IDEA

To a transformer, an image is just more tokens: patches projected into the same vector space as words.

Check 1: From pixels to tokens

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

How many parameters does a linear layer from 192 to 128 numbers have (with biases)?

.....

1.2 CALCULATION

How many 8×8 patch tokens does a 64×64 image make?

.....

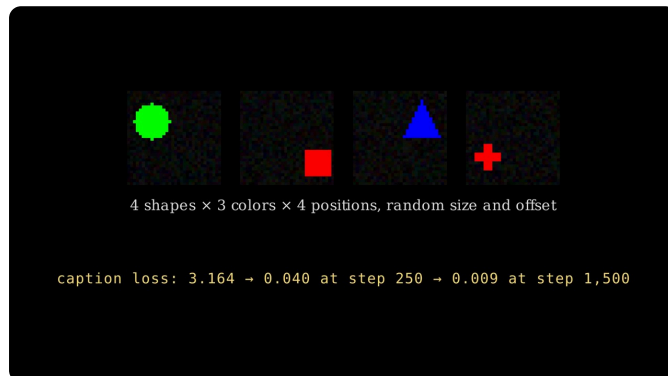
CONCEPT 2 OF 3

Training and testing

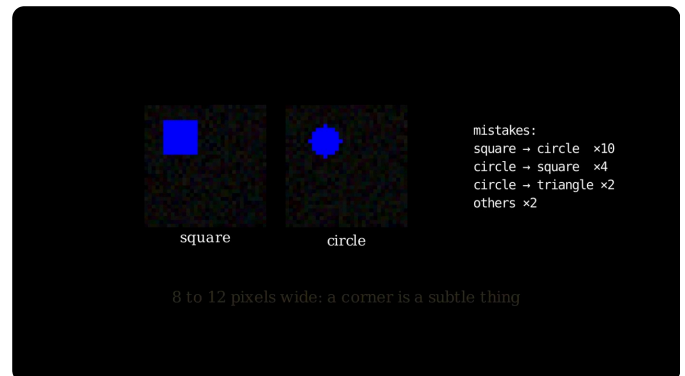
▶ Video 0:55–1:29

The model trains on random pictures of circles, squares, triangles and crosses, in three colors and four positions, with random size and offset. The caption loss falls from **3.164** to **0.040** in 250 steps (0.009 at step 1,500).

On 200 new images: color right **100%** of the time, position **100%**, shape **91%**. Most shape mistakes are squares called circles (10) and circles called squares (4), with two circles called triangles and two other swaps. These shapes are only 8 to 12 pixels wide, and at that size a corner is a subtle thing.



▶ 1:08 Training images: 4 shapes × 3 colors × 4 positions; caption loss 3.164 → 0.040 by step 250.



▶ 1:27 Most mistakes: squares called circles (10) and circles called squares (4).

KEY IDEA

Coarse properties (color, position) are easy from patches; fine shape details need enough resolution.

Check 2: Training and testing

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 MULTIPLE CHOICE

Which property did the model get wrong most often?

- A. Color
- B. Position
- C. Shape
- D. None

2.2 SHORT ANSWER

Why might color be easier than shape for this model?

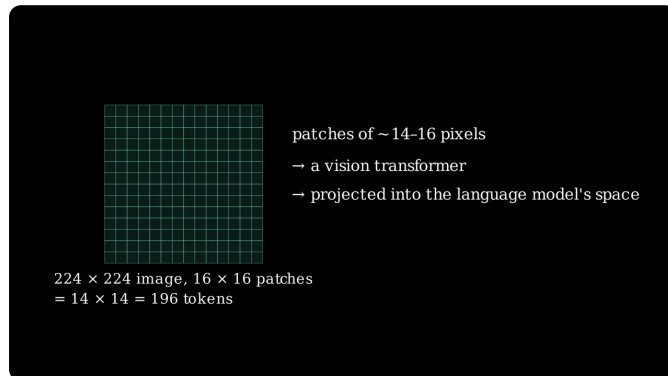
.....

CONCEPT 3 OF 3

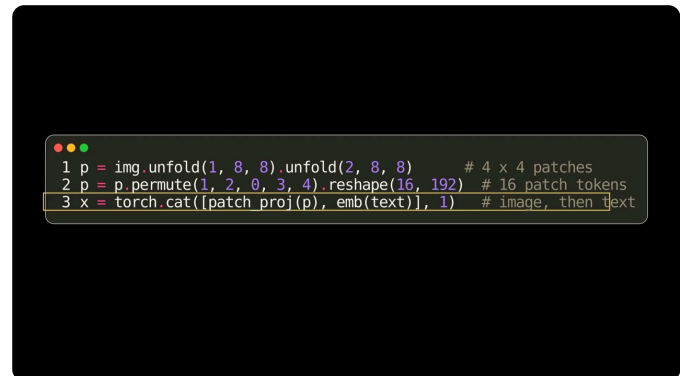
Real models, and the cost

▶ Video 1:29–2:08

Real vision-language models follow the same plan at scale: the image is cut into patches of around 14 or 16 pixels, passed through a vision transformer, and projected into the language model's space. A 224×224 image in 16×16 patches is $14 \times 14 = \mathbf{196 \text{ tokens}}$. The catch: images cost tokens. A detailed, high-resolution image can take hundreds or thousands, competing with text for the context window.



▶ 1:47 Real models: ~14-16-pixel patches, a vision transformer, projection into the language model; $224 \times 224 \rightarrow 196$ tokens.



▶ 2:06 Three steps in code: cut into patches, project, concatenate with the text.

KEY IDEA

The number of image tokens grows with the square of the resolution divided by the patch size.

Check 3: Real models, and the cost

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

How many tokens does a 336×336 image make with 14×14 patches? And 448×448 ?

3.2 HANDS-ON CODE

Try it yourself. Check the patch shapes with the episode's code.

```
img = draw("cross", "red", "bottom left")
print(img.shape, patches(img).shape)
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. From pixels to tokens · rewatch ▶ 0:08-0:55

1.1 24,704. 192×128 weights + 128 biases.

1.2 64. $(64 / 8)^2 = 8 \times 8$.

2. Training and testing · rewatch ▶ 0:55-1:29

2.1 C. Shape 91% vs 100% for color and position.

2.2 Color is visible in any single pixel of the shape, so one patch token carries it; telling a square from a circle depends on a few corner pixels. The mistakes were almost all between the two most similar shapes.

3. Real models, and the cost · rewatch ▶ 1:29-2:08

3.1 576 and 1,024. $(336 / 14)^2 = 24^2$ and $(448 / 14)^2 = 32^2$.

3.2 `torch.Size([3, 32, 32])` and `torch.Size([16, 192])`. 3 color channels of 32×32 pixels become 16 tokens of 192 numbers.